Gift Shop Build Plan

Oct 11, 2026 · @chathura

Overview

The project splits into 12 parts in three phases: build and launch a working shop (Parts 1–7), add the agent platform and customer-facing agents (Parts 8–10), then automate operations (Parts 11–12). The shop goes live and takes real orders before any agent is added, so the agents are built against real data.

Stack: FastAPI (Python), SQLAlchemy 2.0 + Alembic, PostgreSQL with pgvector, Redis + ARQ or Celery for background jobs, Next.js for the storefront and the ops console, the Anthropic Python SDK for agents (no LangGraph), a hosted payment gateway, and S3-compatible storage for images.

Scope assumptions: about 100 products, stock held and shipped by you, one main developer.

Principles that apply to every part:

- Each part ends with something that runs, is tested and is merged, never a half-built layer.
- Business logic lives once, in `services/`. API routes, the ops console and agent tools all call the same services.
- Agents never touch the database directly; permissions and approvals are enforced in code, not in prompts.
- Card details never touch your servers: checkout uses the gateway's hosted page.

The 12 parts

Each part lists its goal, what gets built, and the check that says it is done. Durations assume one full-time developer.

Part 1 — Foundations and dev setup (~1 week)

Goal: a repository where everything runs locally with one command.

- Monorepo: `backend/` (FastAPI), `storefront/` and `console/` (Next.js).
- Docker Compose with Postgres + pgvector, Redis and the API.

- Settings via environment variables, linting, formatting, pytest, and CI that runs tests on every push.

Done when: `docker compose up` starts the stack and CI passes on a hello-world endpoint.

Part 2 — Data model, auth and core API (~1.5 weeks)

Goal: the database and the security model everything else builds on.

- SQLAlchemy models and Alembic migrations for customers, staff, products, inventory, orders, order items, addresses, suppliers and purchase orders.
- Customer accounts (email login) and staff accounts with roles: admin, packer, support.
- The `services/` layer pattern and consistent API errors.

Done when: migrations run cleanly, and role checks are covered by tests (a packer cannot change prices).

Part 3 — Catalog and inventory (~1 week)

Goal: your 100 products live in the system with accurate stock.

- Product CRUD with images uploaded to S3-compatible storage; occasion and recipient tags.
- Inventory: stock on hand, reserved stock, reorder point, bin location, and an adjustment log.
- A seed script that imports your existing product data.

Done when: all 100 products are imported with stock levels, and every stock change is logged.

Part 4 — Cart, checkout and payments (~2 weeks)

Goal: a customer can pay and an order is created reliably.

- Cart (guest and logged-in), gift wrap and gift message options, delivery fee rules.
- Order state machine: pending → paid → packing → shipped → delivered, plus cancelled and refunded.
- Hosted payment checkout and a webhook handler that is idempotent (safe if the gateway sends it twice).
- Stock reserved at checkout and released if payment fails or times out.

- Order confirmation emails.

Done when: a test payment creates exactly one paid order, reduces stock, and sends an email.

Part 5 — Storefront (~2 weeks)

Goal: the customer-facing site in Next.js.

- Home, category and occasion pages, product pages, keyword search with filters (price, occasion, recipient).
- Cart, checkout hand-off to the payment page, order confirmation.
- Customer account: order history, tracking, addresses.
- Mobile-first layout, SEO basics (metadata, sitemap), fast image loading.

Done when: a customer can find a gift on a phone, pay, and track the order end to end.

Part 6 — Ops console and fulfillment (~2 weeks)

Goal: you and your staff can run daily operations from one screen.

- Order queue with filters, order detail, status changes, cancellations and manual refunds.
- Pick-and-pack lists and printable packing slips with gift messages.
- Courier integration: create shipments, print labels or record tracking numbers, notify customers.
- Product and stock editing, stock counts, low-stock view, supplier and purchase order records.
- A simple daily sales summary.

Done when: a paid order can go from queue to shipped with tracking without touching the database.

Part 7 — Launch: hardening and deployment (~1 week)

Goal: the shop is live and selling, operated manually.

- Deploy API, workers, database and both frontends; HTTPS, automated backups, error tracking and uptime alerts.
- Security pass: rate limits, input validation, secrets management, admin protected by strong auth.

- Policy pages: shipping, returns, privacy, terms.

Done when: real customers place real orders and you can restore the database from a backup.

Part 8 — Agent harness (~1.5 weeks)

Goal: your own agent framework, built from `agent_core.py`, before any agent goes live.

- Agent loop, tool registry with typed inputs and risk levels (auto, notify, approve), actor-scoped context.
- Approval queue and audit log tables; limits on steps, tokens and cost per run.
- Tracing of every model and tool call, conversation history storage, streaming support.
- Eval runner: scripted scenarios per agent with expected tool calls and outcomes, run in CI.

Done when: a test agent passes its eval suite, and a tool marked “approve” never runs without approval.

Part 9 — Knowledge base (RAG) (~1 week)

Goal: agents can look up accurate facts about products and policies.

- Embeddings for products, stored in pgvector, combined with structured filters (in stock, price, tags).
- Chunking and indexing of policies, FAQs, gift guides and supplier documents.
- Hybrid search (vector + keyword) exposed as `search_products` and `search_knowledge` tools.
- Automatic re-indexing when a product or policy changes.

Done when: a set of test questions returns the right products and policy passages, measured, not eyeballed.

Part 10 — Customer agents (~2 weeks)

Goal: AI help on the storefront that only works from real data.

- Gift concierge: asks about recipient, occasion and budget, recommends in-stock products, adds them to the cart.
- Support agent: order status and tracking, delivery questions, starting returns; read-only order tools at first.

- Gift message helper on the cart page.
- Streaming chat widget in Next.js, with a clear handoff to a human (email or WhatsApp).
- Evals covering prompt injection, other customers' orders and out-of-stock items.

Done when: agents pass their evals, and conversations and costs are visible in tracing.

Part 11 — Approval inbox and ops agents (~2.5 weeks)

Goal: agents do routine operations; you approve the risky ones.

- Console pages: approval inbox (approve, edit, reject), audit log viewer, agent traces and costs, and a switch to pause any agent.
- Event triggers through the job queue: `order.paid`, `stock.low`, `shipment.delayed`, daily schedule.
- Order ops agent: checks new orders, flags suspicious ones, prepares pick lists, books courier shipments.
- Inventory agent: watches reorder points and drafts purchase orders for approval.
- Reporting agent: daily summary of sales, issues and items waiting for you.
- Support agent gains refunds and address changes, routed through approvals.

Done when: a normal day's orders are processed by agents, with every write action logged and risky ones approved.

Part 12 — Growth agents and more autonomy (ongoing, ~2 weeks to start)

Goal: agents help grow the business, and autonomy expands where it has been earned.

- Catalog agent: drafts product descriptions, SEO text and tags for new products.
- Marketing agent: drafts occasion campaigns and abandoned-cart emails for approval.
- Demand forecasting before major gifting seasons, feeding the inventory agent.
- Review audit logs monthly and move proven actions from "approve" to "notify" or "auto".

Done when: each new agent has evals and an autonomy level backed by its track record.

Build order and timeline

At one full-time developer, the shop is live in about 11 weeks and ops automation starts around week 17.

The shop launches in week 11, before any agent is added



build roadmap · 3 phases, 2 gates

Each gate is a checkpoint: don't start the next phase until its criteria hold. Durations are estimates and will shift once Part 1 shows your real pace.

Working rules

Build one part at a time, and don't start the next until the current part's "done when" check passes.

- Start each part with a short spec: the endpoints, tables, screens or agent tools it adds.
- Write tests alongside the code; agent parts also need evals before they go live.
- Keep a running `DECISIONS.md` for choices you may revisit (payment gateway, courier, job queue).
- Demo each finished part to yourself as a customer or staff member would use it.
- Parts 5 and 6 can be built in parallel if a second developer joins; everything else is in order.